

文章笔记

Agentic Memory: A Detailed Breakdown

基于公开课程资料整理

2026 年 3 月 29 日

原文作者: ramakrushna (techwith_ram)

发布日期: 2026 年 3 月 27 日

内容形式: X Article

原文链接: https://x.com/techwith_ram/status/2037499938574110770

项目主页: <https://github.com/hqhq1025/ai-course-notes>

目录

1 引言：为什么 Agent 需要记忆？	2
1.1 本章小结	2
2 什么是 Agentic Memory？	2
2.1 本章小结	2
3 四种记忆类型	2
3.1 In-context Memory（上下文内记忆）	3
3.2 External Memory（外部记忆）	3
3.3 Episodic Memory（情景记忆）	3
3.4 Semantic/Parametric Memory（语义/参数记忆）	4
3.5 本章小结	4
4 记忆在 Agent Loop 中的流转	4
4.1 本章小结	5
5 构建记忆层：实现要点	5
5.1 MemoryStore 类	5
5.2 EpisodicLogger 类	5
5.3 本章小结	6
6 Vector Database 与相似度搜索	6
6.1 相似度搜索原理	6
6.2 本章小结	7
7 记忆管理：遗忘策略	7
7.1 Time-based Decay（基于时间的衰减）	7
7.2 Importance Scoring（写入时重要性评分）	7
7.3 Periodic Consolidation（定期合并）	7
7.4 本章小结	8
8 总结与延伸	8
8.1 核心要点回顾	8
8.2 延伸思考	8

1 引言：为什么 Agent 需要记忆？

大多数 LLM 的交互是**无状态的**——每一次新对话都是一张白纸。模型不知道你是谁、之前聊过什么、一起构建了什么。对于简单的聊天机器人，这无伤大雅；但对于一个需要执行任务、做出决策并随时间不断改进的 **Agent** 来说，这种”失忆症”是致命的。

记忆是 Agent 进化的关键

真正的智能不仅仅是”回答得好”，而是能够**记住、学习并在已有经验上持续构建**。记忆将一个无状态的系统转变为一个能够真正进化的系统。

1.1 本章小结

LLM 天生无状态，每次对话都从零开始。Agent 需要记忆系统来维持上下文、积累经验和持续学习。

2 什么是 Agentic Memory？

Agentic Memory 不是单一组件，而是一个在后台运作的**完整系统**——包括不同类型的存储、信息检索方式以及智能管理策略，使 Agent 能够跨时间维持上下文。

记忆系统同时承担三个截然不同的职责：

1. **Continuity (连续性)** —— 关乎身份认同。Agent 知道你是谁、你的偏好是什么、你们之前一起构建了什么。没有连续性，每次交互都像从头开始。
2. **Context (上下文)** —— 关乎当前任务。刚刚发生了什么、调用了哪个工具、返回了什么结果、下一步该做什么。它是多步工作流不崩溃的保障。
3. **Learning (学习)** —— 关乎持续进步。理解什么有效、什么无效，并在长期中逐步改进决策，而非重复犯同样的错误。

三大职责的协同

一个设计良好的 Agent 记忆系统同时处理以上三个方面，为每个方面使用不同的存储后端。三者协同让 Agent 表现得一致、可靠，且在每次交互后都变得更聪明。

2.1 本章小结

Agentic Memory 是一个多层系统，同时服务于身份连续性、任务上下文和经验学习三个目标。

3 四种记忆类型

该领域已收敛到四种不同的记忆类型，可以类比为大脑的四个不同区域，各自为特定任务而设计。

3.1 In-context Memory (上下文内记忆)

Context Window 是 Agent 的**工作台**——台面上的一切都可以即时访问，模型可以在单次 forward pass 中对其进行推理，无需检索步骤。

但工作台有大小限制：每个 token 都消耗算力和金钱，session 结束后台面会被清空。

上下文中通常包含以下内容：

- **System Prompt**：Agent 的人设、规则、能力描述、当前日期/用户信息
- **Conversation History**：本次 session 中的对话往来
- **Tool Call Results**：Agent 刚刚调用工具的输出
- **Retrieved Memories**：从外部存储拉取的记忆片段
- **Scratchpad**：中间推理过程（Chain-of-Thought 输出）

Sliding Window 问题

在长对话中，历史消息不断累积，最终会溢出 context 上限。简单截断最早消息会丢失重要的早期上下文。应采取更好的策略：

- **Summarization**：定期将旧对话压缩为简短摘要
- **Selective Retention**：保留包含关键事实、决策或工具结果的对话轮次，丢弃闲聊
- **Offload to External Memory**：将重要事实提取到 Vector Store，按需检索

3.2 External Memory (外部记忆)

External Memory 是持久化在模型之外的一切——数据库、向量存储、键值存储和文件。它能跨 session 存活，如果存储得当，Agent 可以回忆起六个月前的事情。

外部存储有两种风格：

- **Structured Store (精确查找)**：PostgreSQL、Redis、SQLite。通过 key、ID 或 SQL 查询。快速、可预测，适合用户画像、偏好和结构化数据。
- **Vector Store (语义搜索)**：Pinecone、Chroma、pgvector。通过语义查询——”找到与这个概念相似的记忆”。对非结构化笔记和 Episodic Recall 至关重要。

检索设计是核心

检索步骤是记忆系统的瓶颈。如果你没有检索到正确的记忆，Agent 就像这些记忆根本不存在一样。好的记忆架构是 **20% 存储 + 80% 检索设计**。

3.3 Episodic Memory (情景记忆)

Episodic Memory 是最被低估的记忆类型。外部记忆存储**事实**，而情景记忆存储**事件**——特别是过去行动的结果。

最简单的形式是一个结构化日志：每当 Agent 完成一个任务，它就记录发生了什么。随着时间推移，这个日志变成了一个丰富的自我认知来源，Agent 可以在做决策前查阅它。

一个 Episode 的典型结构：

- **Task**: 任务描述
- **Strategy**: 采用的策略
- **Outcome**: 执行结果（成功/失败）
- **Lesson**: 学到的经验教训

Reflection Loop (反思循环)

当新任务到来时，Agent 检索语义上最相似的历史 Episode，用它们来选择策略。这本质上是从小历史中进行 Few-shot Learning，而不是从手工构造的数据集。

3.4 Semantic/Parametric Memory (语义/参数记忆)

这是模型“与生俱来”的记忆——在训练过程中编码到权重里的一切：世界知识、语言模式、推理策略、编码规范和文化常识。

它始终在那里，Agent 永远不需要检索它。但它有严格的限制：

- **冻结在训练时刻**：模型不知道 cutoff date 之后发生的事
- **运行时无法更新**：不重新训练或微调就无法注入新的永久事实
- **不透明**：你无法精确检查模型“知道”什么或不知道什么
- **容易产生幻觉**：模型用看似合理但实际错误的内容填补知识空白

不要依赖参数记忆处理敏感信息

对于时间敏感、领域特定或私有的信息，不要依赖 Parametric Memory，应使用外部检索。Parametric Memory 只是在没有更好来源时，作为通用世界知识的兜底方案。

正确的思维模型

Parametric Memory 是 Agent 的**通识教育**；External、Episodic 和 In-context Memory 是 Agent 的**在职经验**。最好的 Agent 将两者结合使用。

3.5 本章小结

四种记忆类型各司其职：In-context Memory 提供即时推理能力，External Memory 提供跨 session 持久化，Episodic Memory 记录行动结果以支持学习，Parametric Memory 提供通用基础知识。

4 记忆在 Agent Loop 中的流转

每当 Agent 处理一个请求时，记忆系统贯穿整个流程。核心机制是：记忆操作**环绕** LLM 调用——调用前检索，调用后写入。

典型的 Agent Loop 流程如下：

1. 用户发送请求

2. **Memory Retrieval**: 从 External Memory 和 Episodic Memory 中检索相关记忆
3. 将检索结果注入 Context Window (In-context Memory)
4. LLM 基于完整上下文进行推理并生成响应
5. **Memory Write**: 将本次交互中的关键信息写入 External Memory 和 Episodic Memory
6. 返回响应给用户

模型本身是无状态的

模型本身是无状态的；记忆系统才是制造出“有状态、有意识的 Agent”这一幻象的关键。

4.1 本章小结

Agent Loop 中记忆操作在 LLM 调用前后进行——先检索、后写入。模型只是中间的推理引擎，记忆系统赋予它状态感。

5 构建记忆层：实现要点

原文以 Python + OpenAI Embeddings + ChromaDB 为例，展示了如何构建一个记忆层。核心组件包括两个类。

5.1 MemoryStore 类

负责记忆的写入（生成 Embedding 后存储）和语义检索，是整个系统的基础。

```
1 class MemoryStore:
2     def __init__(self, collection_name):
3         self.client = chromadb.Client()
4         self.collection = self.client.get_or_create_collection(
5             collection_name)
6         self.embedder = OpenAI()
7
8     def add(self, text, metadata=None):
9         embedding = self.embedder.embed(text)
10        self.collection.add(embedding, text, metadata)
11
12    def search(self, query, top_k=5):
13        query_embedding = self.embedder.embed(query)
14        return self.collection.query(query_embedding, n=top_k)
```

Listing 1: MemoryStore 核心接口（伪代码）

5.2 EpisodicLogger 类

在 MemoryStore 之上添加情景日志层，记录每次任务的执行过程和结果。

```

1 class EpisodicLogger:
2     def __init__(self, memory_store):
3         self.store = memory_store
4
5     def log_episode(self, task, strategy, outcome, lesson):
6         episode = {
7             "task": task, "strategy": strategy,
8             "outcome": outcome, "lesson": lesson,
9             "timestamp": now()
10        }
11        self.store.add(json.dumps(episode),
12                       metadata={"type": "episode"})
13
14    def recall_similar(self, task_description, top_k=3):
15        return self.store.search(task_description, top_k)

```

Listing 2: EpisodicLogger 核心接口（伪代码）

5.3 本章小结

记忆层的核心是 MemoryStore（向量存储 + 语义检索）和 EpisodicLogger（行动日志 + 经验回溯）。两者搭配即可构建一个具备基本记忆能力的 Agent。

6 Vector Database 与相似度搜索

Vector Database 是任何严肃记忆系统的核心。与 SQL 的精确匹配不同，它在高维空间中找到一个向量的最近邻居，实现**语义搜索**——即使没有共享词汇也能找到概念相关的记忆。

6.1 相似度搜索原理

每条记忆被转换为一个向量（以 OpenAI Embedding 模型为例，是一个包含 1,536 个浮点数的数组）。概念相似的文本会产生相似的向量。查询时，将查询文本也嵌入为向量，然后通过 Cosine Similarity 找到最近的向量。

$$\text{cosine_similarity}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \cdot \|\mathbf{b}\|}$$

其中：

- $\mathbf{a}, \mathbf{b}$: 两个 Embedding 向量
- $\mathbf{a} \cdot \mathbf{b}$: 向量点积
- $\|\mathbf{a}\|, \|\mathbf{b}\|$: 向量的 L2 范数

Vector Database 选型建议

- **ChromaDB**: 本地开发首选, 轻量易上手
- **pgvector**: 如果你已在用 PostgreSQL, 零额外基础设施
- **Pinecone / Qdrant**: 需要大规模部署时使用

6.2 本章小结

Vector Database 通过将文本转为高维向量并计算 Cosine Similarity 来实现语义搜索。选择工具时应根据开发阶段和规模需求决定。

7 记忆管理：遗忘策略

真正的记忆系统不只是不断积累, 还需要**精心筛选**。一个不断增长、缺乏重点的存储会随时间退化——检索噪声增大、延迟攀升、矛盾的记忆让 Agent 困惑。

无限堆积记忆的陷阱

没有遗忘策略的记忆系统最终会变得**比没有记忆更糟糕**: 检索噪声淹没有效信息, 互相矛盾的记忆导致 Agent 决策混乱。

7.1 Time-based Decay (基于时间的衰减)

较旧的记忆通常相关性较低。通过 recency 和 semantic relevance 的组合来评分记忆。研究中常用的公式:

$$\text{score} = \alpha \cdot \text{relevance} + (1 - \alpha) \cdot \text{recency_decay}(t)$$

其中:

- α : 语义相关性与时间衰减之间的权衡系数
- relevance: 查询与记忆之间的语义相似度
- $\text{recency_decay}(t)$: 基于时间 t 的衰减函数 (如指数衰减)

7.2 Importance Scoring (写入时重要性评分)

存储记忆时, 让模型对自己的输出进行重要性评分, 只存储高分项。这从源头过滤噪声。

7.3 Periodic Consolidation (定期合并)

运行定期任务 (如每天夜间), 将重复或高度相似的记忆合并为一条规范的摘要。这类似于人类睡眠过程中的**记忆巩固**机制。

类比人类记忆

人类大脑在睡眠中进行”记忆巩固”——将白天零散的短期记忆整理、合并为长期记忆。Agent 的 Periodic Consolidation 机制借鉴了同样的原理：定期清理冗余，保留精华。

7.4 本章小结

三种遗忘策略各有用途：Time-based Decay 让旧记忆自然淡出，Importance Scoring 在写入时过滤噪声，Periodic Consolidation 定期合并重复记忆。三者结合可维持高质量的记忆库。

8 总结与延伸

8.1 核心要点回顾

- 记忆是让 AI Agent 从”工具”升级为”伙伴”的关键能力
- Agentic Memory 同时服务于三个目标：连续性、上下文和学习
- 四种记忆类型（In-context、External、Episodic、Parametric）各司其职，协同工作
- 记忆架构的核心在于**检索设计**而非存储
- 记忆管理（遗忘策略）与记忆存储同等重要

8.2 延伸思考

- **MemGPT**：将操作系统的虚拟内存概念引入 LLM Agent，自动在不同层级的记忆之间进行 paging
- **Generative Agents** (Stanford)：在模拟世界中使用 Episodic Memory + Reflection 机制让 Agent 展现出类人的社会行为
- **RAG 与记忆的关系**：Retrieval-Augmented Generation 可以看作 External Memory 的一种应用形态，但完整的 Agentic Memory 还包括写入、遗忘和反思机制
- **Multi-Agent 场景下的共享记忆**：当多个 Agent 协作时，如何设计共享的记忆空间和权限控制是一个开放问题